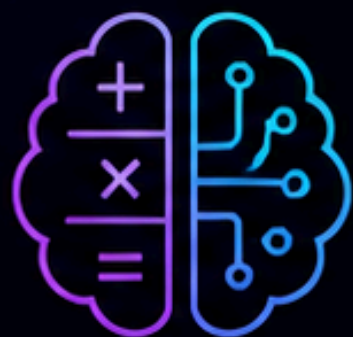




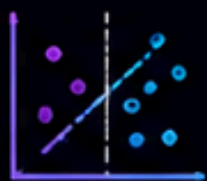
The Math of Categorization



Day 9: Logistic Regression & K-Nearest Neighbours



CLASSIFY



SEPARATE

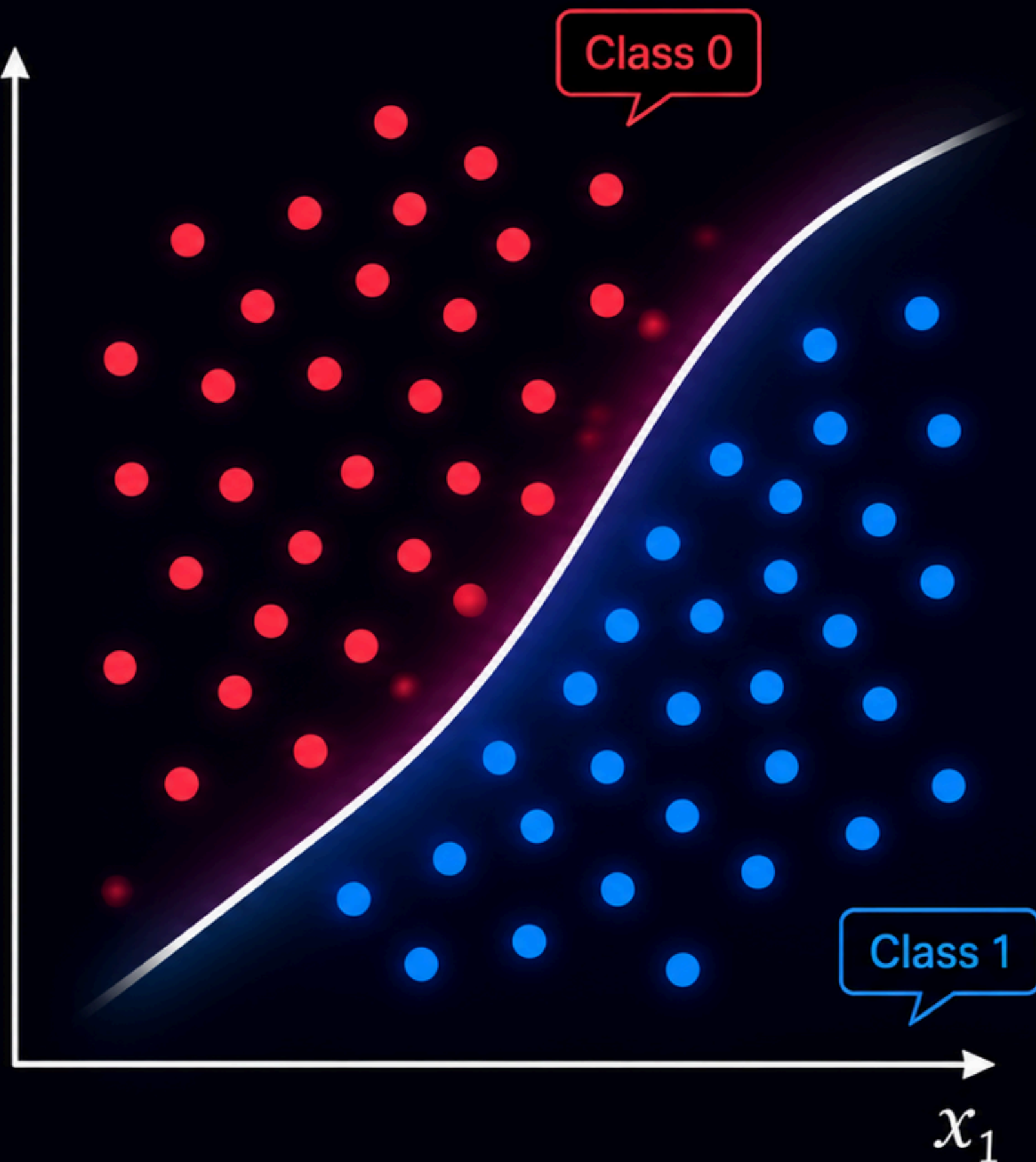


PREDICT



DECIDE

x_2



Class 1

x_1



Predicting Categories



Regression: Predicts **continuous, infinite** values (e.g., predicting an exact score of 98/98).



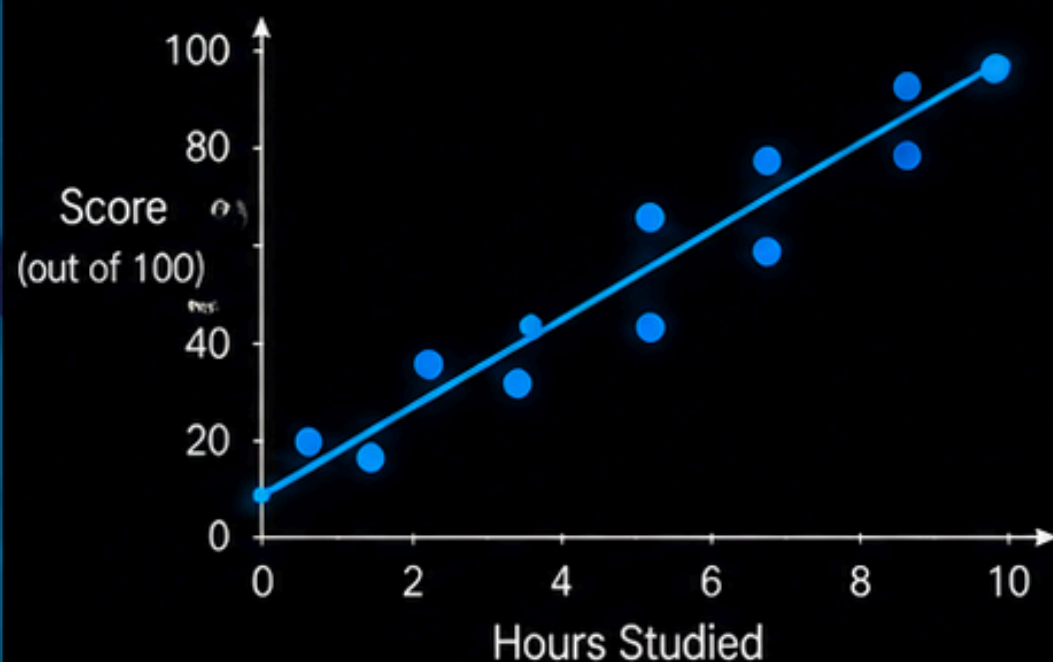
Classification: Predicts **discrete class labels** (e.g., predicting "Pass", "Fail", or "Honors").



Binary Classification (2 classes) vs. **Multi-class Classification** (3+ classes).

REGRESSION

Predicts **continuous, infinite** values



Example: Predict the exact score of a student.

Output: 98.37 / 100

CLASSIFICATION

Predicts **discrete class labels**

PASS



FAIL



HONORS

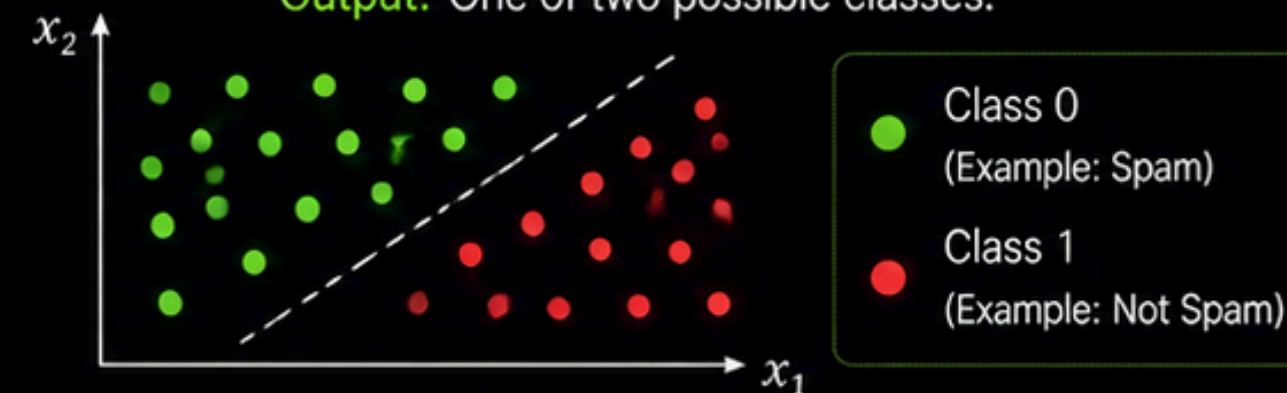


Example: Predict the class of a student.

Output: "Pass" or "Fail" or "Honors"

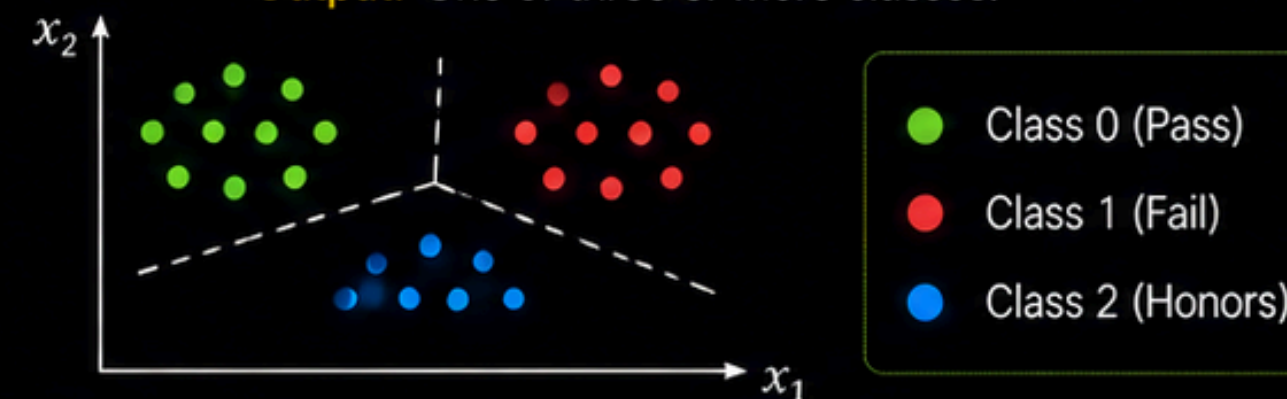
BINARY CLASSIFICATION (2 CLASSES)

Output: One of two possible classes.



MULTI-CLASS CLASSIFICATION (3+ CLASSES)

Output: One of three or more classes.



Key Takeaway: Regression answers "how much?", while Classification answers "which class?"



It's Not Actually Regression



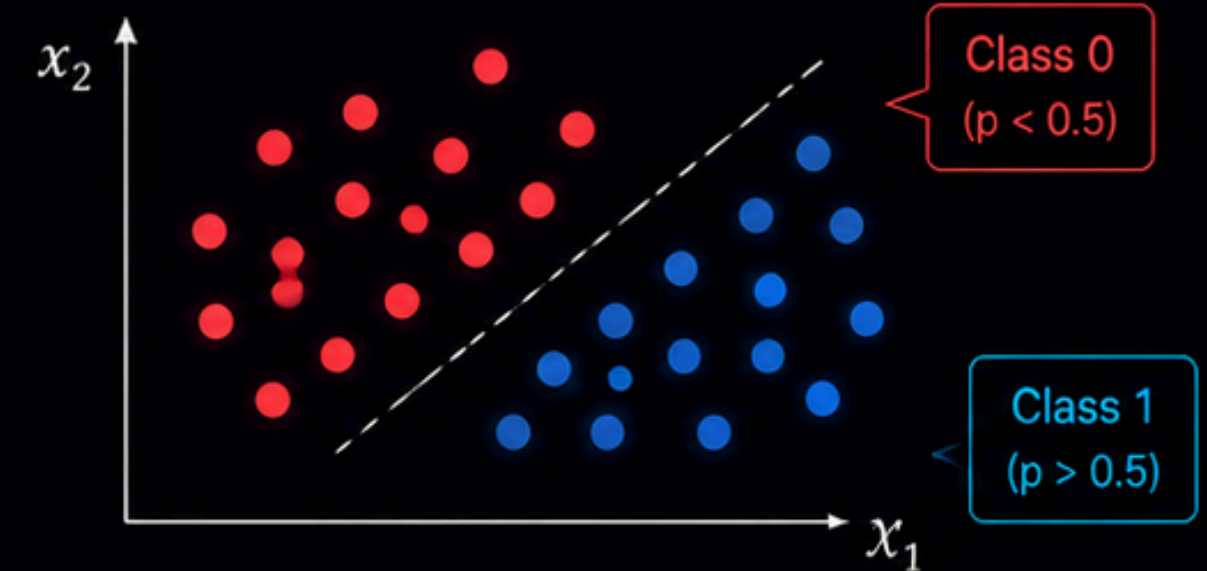
Despite the name, it is a **classification** algorithm.



It calculates the **probability** that a data point belongs to the default class (values between **0.0** and **1.0**).



If the probability is **> 0.5**, it belongs to **Class 1**.
If **< 0.5**, it belongs to **Class 0**.



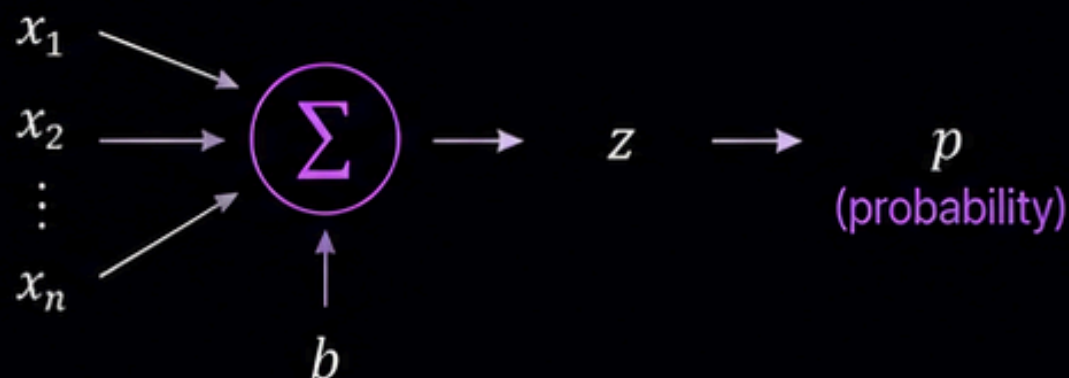
1. FROM LINEAR SCORE TO PROBABILITY

Logistic Regression takes a **linear combination** of features...

$$z = w_1x_1 + w_2x_2 + b$$

(Linear Score)

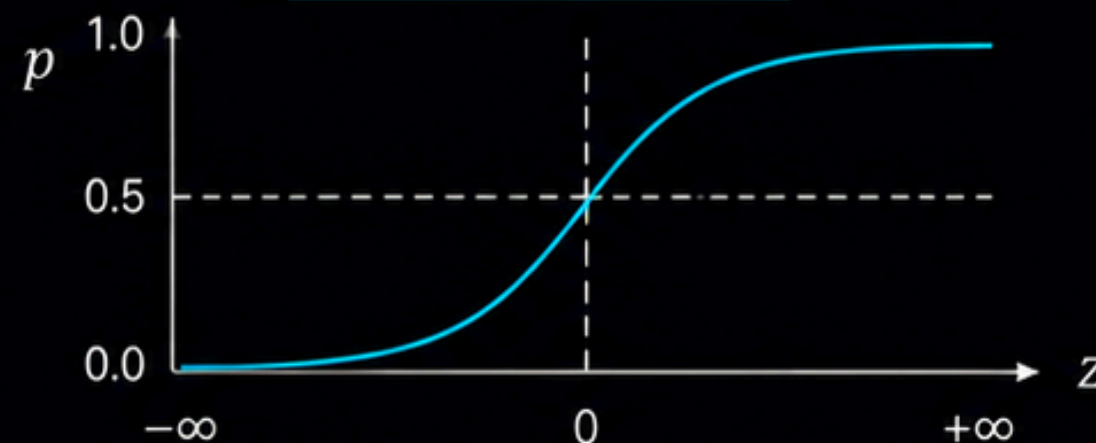
Sigmoid
Function



2. SIGMOID FUNCTION (SQUASHES TO 0-1)

Maps any real-valued score z to a **probability** p between 0 and 1.

$$p = \frac{1}{1 + e^{-z}}$$



3. DECISION RULE

Convert probability to a class label.

$$p > 0.5$$



Class 1
(Positive)

$$p < 0.5$$



Class 0
(Negative)



Threshold (0.5) can be adjusted based on the problem and cost of errors.



Logistic Regression = **Linear model** + **Sigmoid function** + **Threshold**

Output is a **class label**, not a continuous value.



The Sigmoid Function

Squashing the Line



- Standard linear regression ($y = wx + b$) can output values like **-150** or **5000**, which **ruins probability**.



- We pass that linear equation through the **Sigmoid activation function** to force it between **0** and **1**.

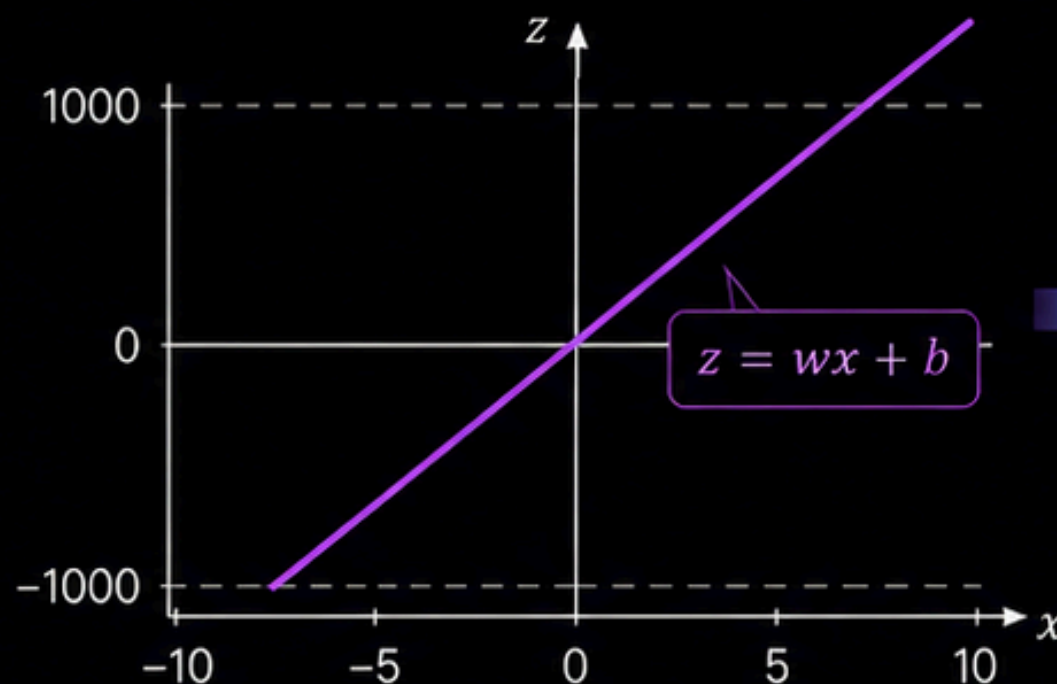


- Formula:**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

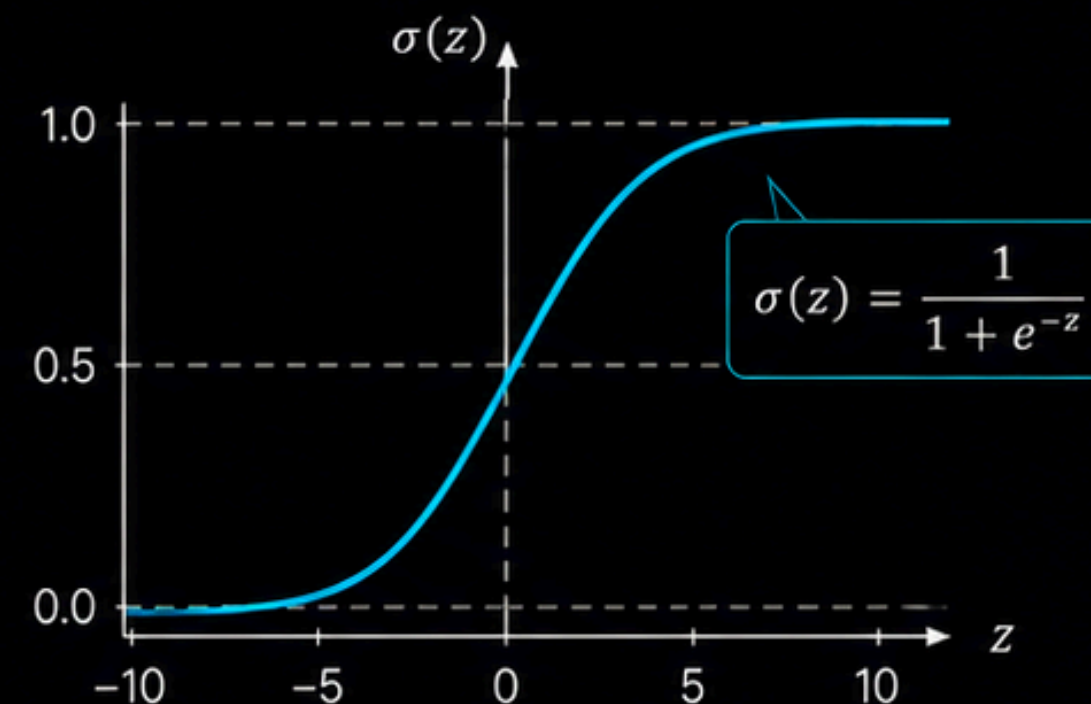
1. LINEAR OUTPUT ($z = wx + b$)

Outputs can be any real number
($-\infty$ to $+\infty$)



2. SIGMOID OUTPUT ($\sigma(z)$)

Squashes any real number
into a probability between **0** and **1**



SOME EXAMPLES

z (linear output)	$\sigma(z)$ (probability)
-10	0.000045
-5	0.0067
0	0.5000
5	0.9933
10	0.99995

Linear output (z)

e.g., -150, 0, 25, 5000
(any real number)



Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Probability ($\sigma(z)$)

e.g., 0.00, 0.50, 0.92, 1.00
(between 0 and 1)



Key Takeaway: The Sigmoid function “squashes” any real-valued output into a **probability** between **0** and **1**, making it perfect for classification.



Log Loss (Binary Cross-Entropy)

How Logistic Models Learn



- We cannot use **Mean Squared Error (MSE)** for Logistic Regression because the Sigmoid curve makes the cost function **non-convex** (lots of local minimums).



- We use **Log Loss**. It heavily penalizes the model for being **confident and wrong**.



- Formula:

$$Loss = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

$y_i \in \{0, 1\}$ = Actual label

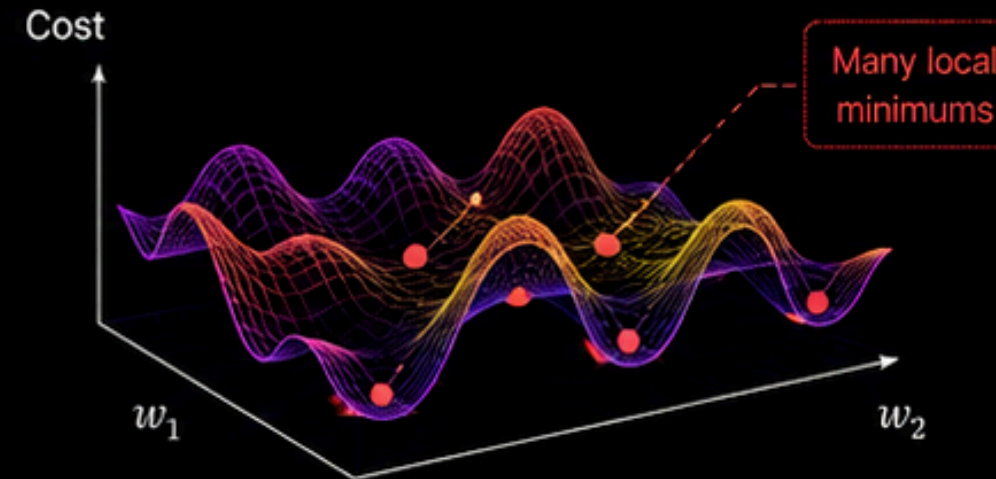
$\hat{y}_i \in (0, 1)$ = Predicted probability (after Sigmoid)

N = Number of samples

WHY NOT MEAN SQUARED ERROR?

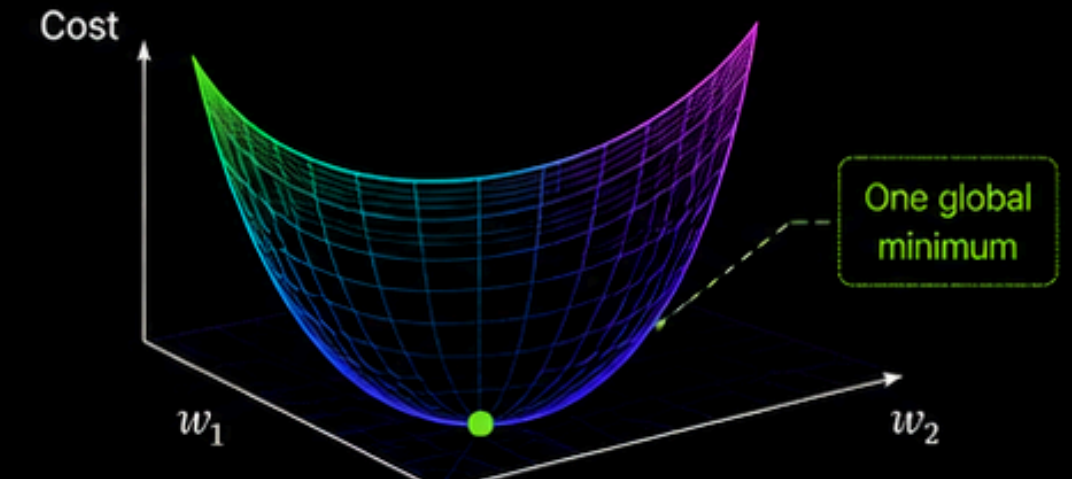
MSE Cost Function (Non-Convex)

Many local minimums make optimization hard.



Log Loss (Convex)

Single global minimum makes optimization easy.



Convex functions (like Log Loss) guarantee we find the **global minimum**.

LOG LOSS INTUITION

Log Loss penalizes confident wrong predictions much more than less confident ones.



SOME EXAMPLES

Actual (y_i)	Predicted Prob. ($\hat{y}_i$)	Log Loss	Interpretation
1	0.95	0.051	Correct and confident → Low loss
1	0.05	2.996	Wrong and confident → Very high loss
0	0.05	0.051	Correct and confident → Low loss
0	0.95	2.996	Wrong and confident → Very high loss

Remember:

Log Loss $\rightarrow \infty$
when we are
confident (close to
0 or 1) and **wrong**.

Log Loss $\rightarrow 0$
when we are
confident and
correct.



Key Takeaway: Log Loss is the right loss function for Logistic Regression because it is **convex**, easy to optimize, and **penalizes confident mistakes strongly**.



Beyond Binary

① *
vs

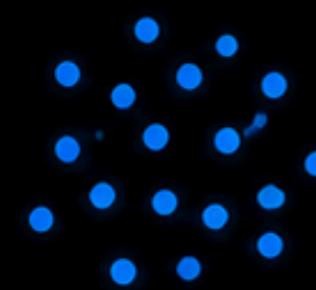
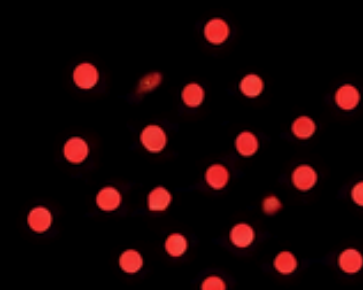
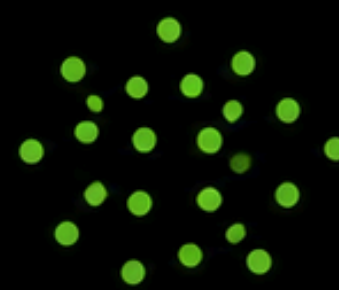
- **One-vs-Rest (OvR):** Trains a separate binary classifier for each class against all others combined.

$\sum$
= 1.0

- **Multinomial (Softmax):** Upgrades the Sigmoid function to output a **probability distribution** across multiple classes that perfectly **sums to 1.0**.

 x_2

Example: 3-Class Classification



- Class 0
- Class 1
- Class 2

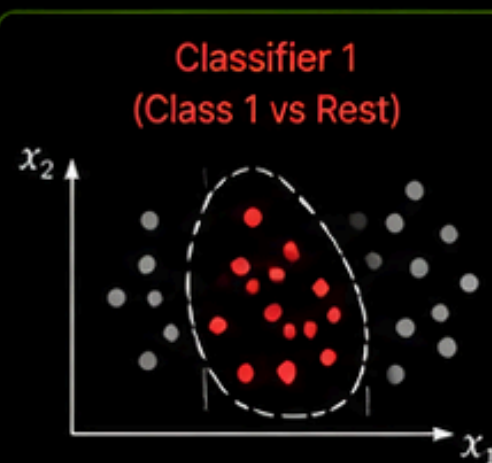
 x_1

① ONE-VS-REST (OvR)

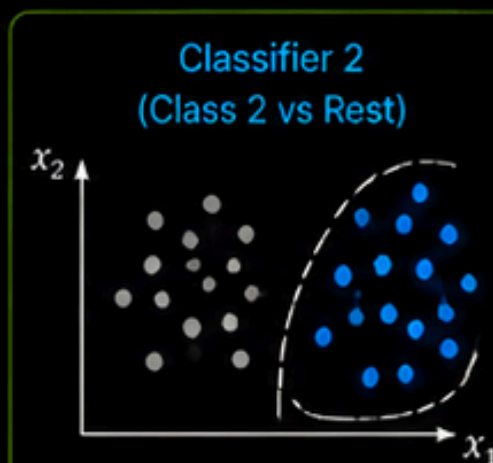
Train **K binary classifiers** (where K = number of classes).
Each classifier separates **one class** from all others.



Positive: Class 0
Negative: Class 1 + Class 2



Positive: Class 1
Negative: Class 0 + Class 2



Positive: Class 2
Negative: Class 0 + Class 1

New
Point



Get **3 probabilities**
(one from each classifier)

Pick the class with
the **highest probability**

② MULTINOMIAL (SOFTMAX)

Single model outputs a **probability distribution** across all classes
using the **Softmax** function.

SOFTMAX FUNCTION

$$P(y = k | x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

where $z_k = w_k^T x + b_k$
K = number of classes

HOW IT WORKS

Input
features (x)

x_1
 x_2
 $\vdots$
 x_n

Linear scores
(z)

z_0
 z_1
 $\vdots$
 z_{K-1}

Softmax
(probabilities)

p_0
 p_1
 $\vdots$
 p_{K-1}

Properties

- $0 \leq p_k \leq 1$ for all k
- $\sum_{k=0}^{K-1} p_k = 1.0$ (perfect probability distribution)

EXAMPLE (3 CLASSES)

Class	Linear Score (z_k)	e^{z_k}	Probability (Softmax)
Class 0	1.20	3.320	0.65
Class 1	0.10	1.105	0.22
Class 2	-0.80	0.449	0.13

$$0.65 + 0.22 + 0.13 = 1.00$$

(sums to 1.0)



OvR is simple and effective. **Softmax (Multinomial)** is more elegant and uses shared information across classes.



K-Nearest Neighbours (KNN)

Lazy Learning



- KNN does absolutely **no mathematical training** upfront. It simply **memorizes** the entire dataset.



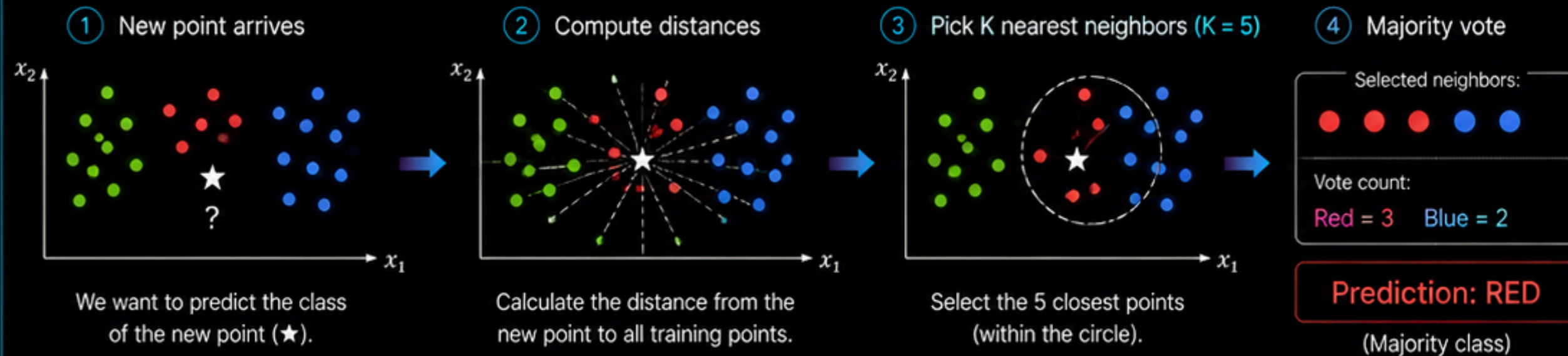
- When a new data point arrives, it looks at the '**K**' **closest** data points in the training set and takes a **majority vote**.



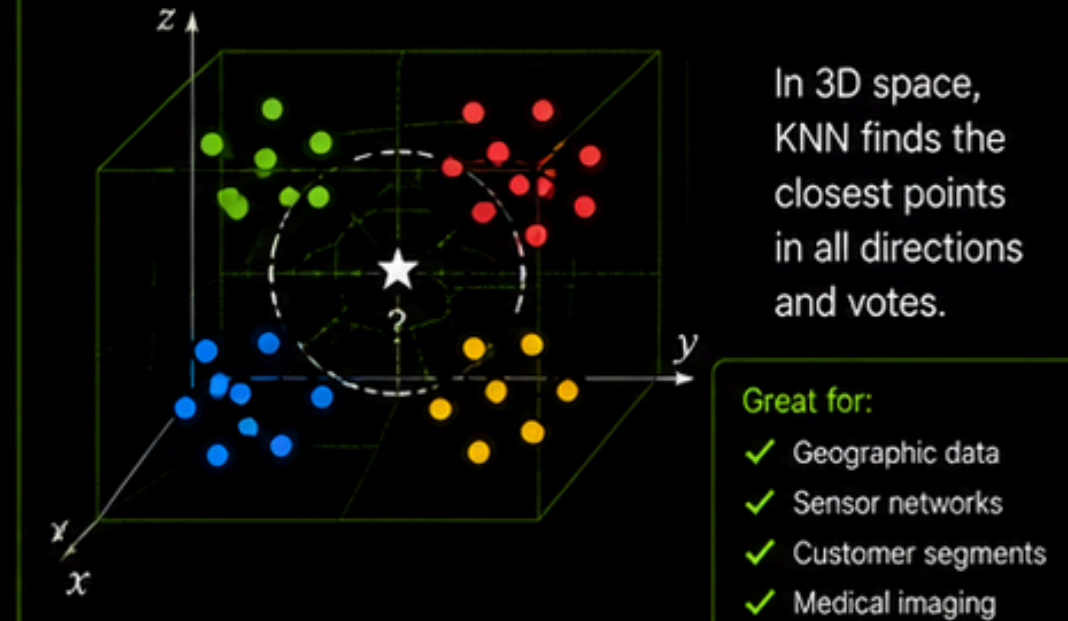
- Ideal for **localized groupings**, like classifying **3D spatial coordinate clusters**.



HOW KNN MAKES A PREDICTION (EXAMPLE WITH K = 5)



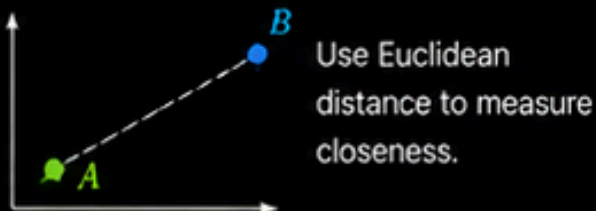
IDEAL FOR 3D SPATIAL CLUSTERS



COMMON DISTANCE METRIC: EUCLIDEAN DISTANCE

For two points $A(x_1, y_1)$ and $B(x_2, y_2)$:

$$d(A, B) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$



CHOOSING K

- ✓ Small K (e.g., 1–3): Captures local patterns, but sensitive to noise.
- ✗ Large K (e.g., 15–30): Smoother decision boundary, but may miss local details.

Choose K using cross-validation.

PROS

- ✓ Simple to understand
- ✓ No training time
- ✓ Works well for non-linear boundaries

CONS

- ✗ Slow prediction for large datasets
- ✗ Needs feature scaling
- ✗ Sensitive to irrelevant features
- ✗ Needs good choice of K

★ **Key Takeaway:** KNN is lazy—no training, just remembering. When in doubt, **look at your neighbors and take a vote!**



Measuring Distance

? How does it define “closest”?



- **Euclidean Distance (L2):**
The straight-line distance.

$$d(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

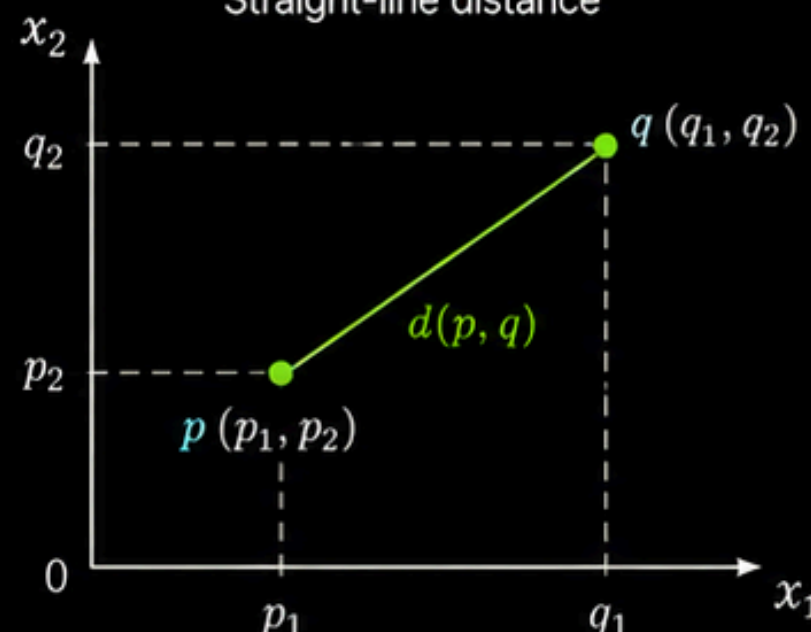


- **Manhattan Distance (L1):**
The grid-like distance.

$$d(p, q) = \sum_{i=1}^n |q_i - p_i|$$

EUCLIDEAN DISTANCE (L2)

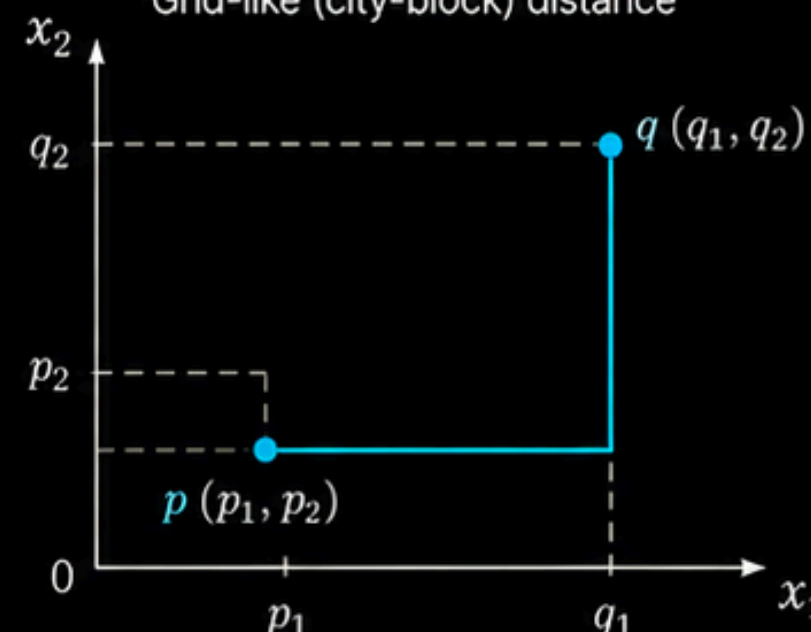
Straight-line distance



The shortest path between two points.

MANHATTAN DISTANCE (L1)

Grid-like (city-block) distance



Moves along axes (no diagonals).

INTUITION



L2 (Euclidean):

- As the crow flies
- Sensitive to large differences in any dimension



L1 (Manhattan):

- Block by block
- More robust to outliers in individual dimensions

EXAMPLE (2D)

Let $p = (1, 1)$ and $q = (4, 5)$

Dimension	p_i	q_i	$ q_i - p_i $	$(q_i - p_i)^2$
1 (x_1)	1	4	$ 4 - 1 = 3$	$(4 - 1)^2 = 9$
2 (x_2)	1	5	$ 5 - 1 = 4$	$(5 - 1)^2 = 16$



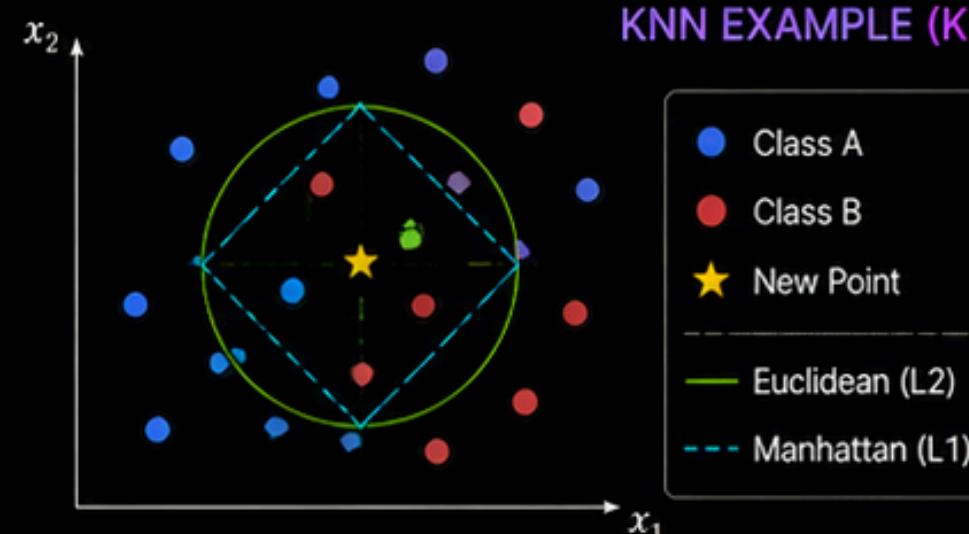
Euclidean Distance (L2)

$$d(p, q) = \sqrt{9 + 16} = \sqrt{25} = 5.00$$

Manhattan Distance (L1)

$$d(p, q) = 3 + 4 = 7.00$$

KNN EXAMPLE (K = 3)



Using Euclidean (L2)

3 closest neighbors



Majority: **Class A**
Prediction: **Class A**

Using Manhattan (L1)

3 closest neighbors



Majority: **Class B**
Prediction: **Class B**

WHEN TO USE WHAT?



Euclidean (L2):

When all features are on similar scales and relationships are continuous.

Manhattan (L1):

When features are in grids/steps or when data has outliers.



Key Takeaway: KNN relies on **distance** to find neighbors. The choice of distance metric changes who is considered “**close**” and can **change the prediction**!



Choosing the 'K' Value

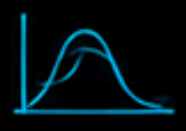
Hyperparameter Tuning

K = 1



- K = 1: Highly unstable, memorizes noise (Overfitting / High Variance).

K = 100



- K = 100: Too smoothed out, misclassifies local nuances (Underfitting / High Bias).

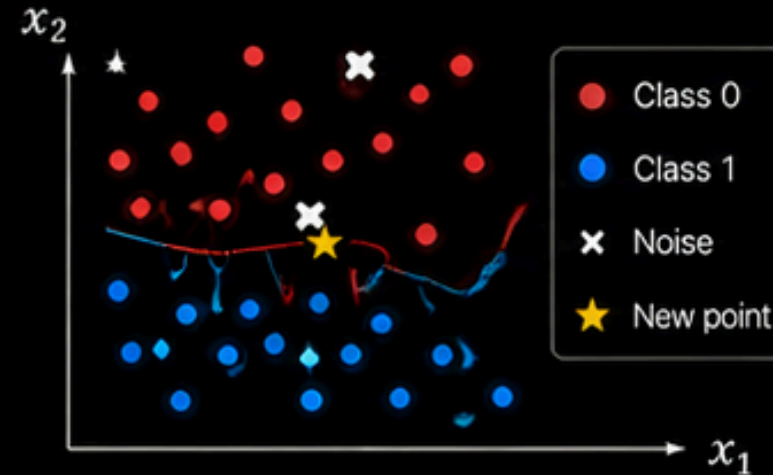


- **Golden Rule:** Always pick an **odd number** for K in binary classification to prevent voting ties.

IMPACT OF DIFFERENT K VALUES (2D EXAMPLE)

K = 1 (Very Small)

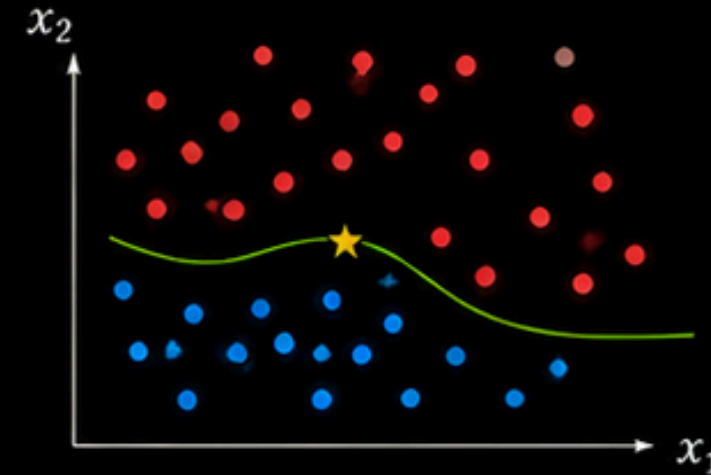
Fits training data almost perfectly, including noise.



Decision boundary is too complex
→ Overfitting (High Variance)

K = 5 (Just Right)

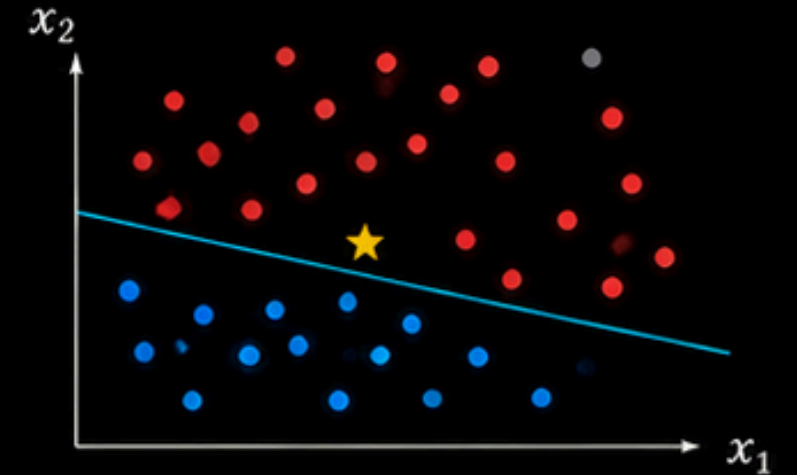
Captures the true pattern without chasing noise.



Good balance
→ Good Generalization

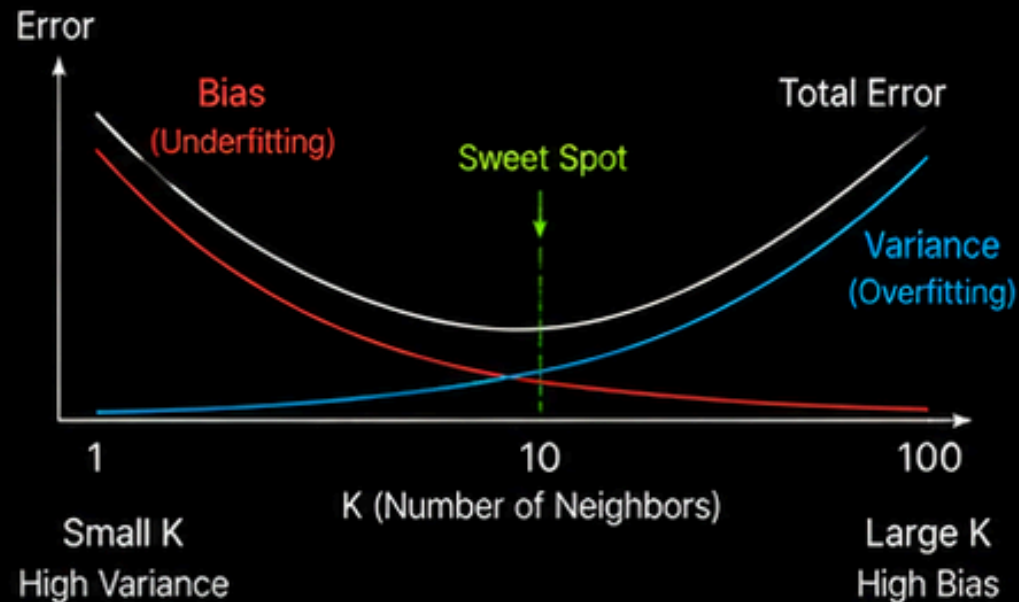
K = 100 (Very Large)

Overly smooth, ignores local structure and nuances.



Decision boundary is too simple
→ Underfitting (High Bias)

BIAS-VARIANCE TRADEOFF



VOTING EXAMPLE (BINARY CLASSIFICATION)

Odd K avoids ties!

K = 3 (Odd) ✓

Neighbors:



Votes → 2 vs 1

Prediction:
Class 0

K = 4 (Even) ✗

Neighbors:



Votes → 2 vs 2

Tie!
Ambiguous

K = 5 (Odd) ✓

Neighbors:



Votes → 3 vs 2

Prediction:
Class 0

HOW TO CHOOSE K?



Use Cross-Validation

Evaluate different K values and pick the one with the lowest validation error.



Start Small, Then Increase

Try K = 1, 3, 5, 7, ... and observe performance.



Consider Data Scale

Larger datasets may benefit from larger K.

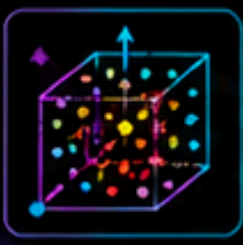


Domain Knowledge

Choose K that aligns with the problem context.

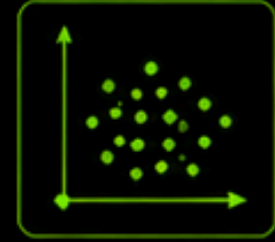


Golden Rule: Always pick an **odd number** for K in binary classification to prevent voting ties.

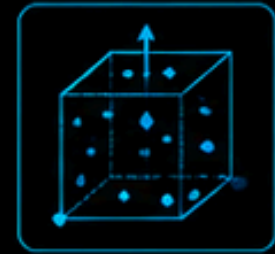


The Curse of Dimensionality

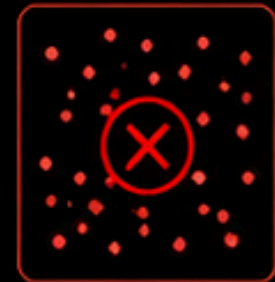
Why KNN Fails on Text and Vision



- In low dimensions (2D/3D), points are tightly packed.



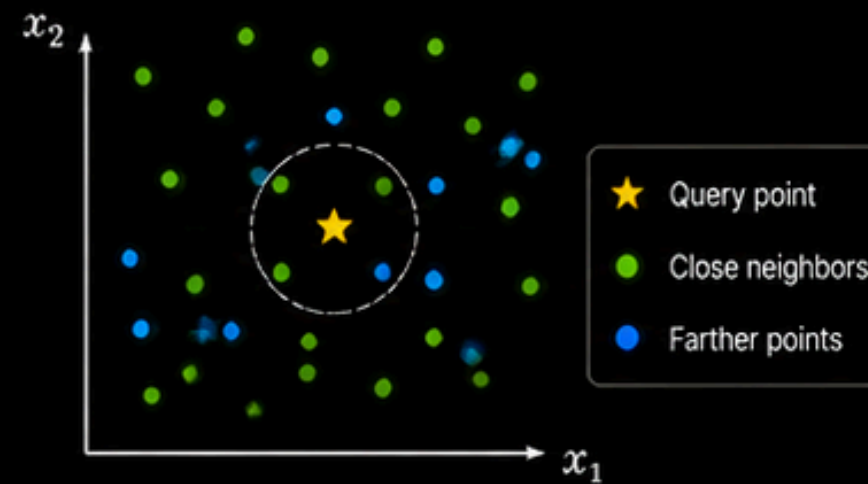
- In high dimensions (e.g., 500-dimension NLP word embeddings), the mathematical volume expands **exponentially**.



- In 500D space, all points are practically the exact same distance apart, rendering **KNN useless**.

LOW DIMENSIONS (2D)

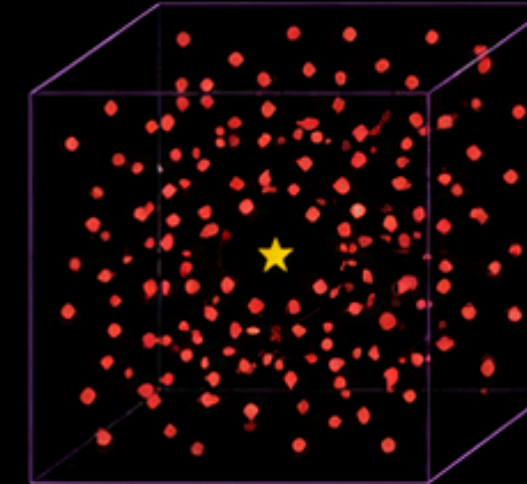
Points are close to meaningful neighbors



Meaningful local neighborhood.
KNN works well.

HIGH DIMENSIONS (500D)

Points look uniformly spread out



No meaningful neighbors.
Everything is far from everything.

VOLUME GROWS EXPONENTIALLY

Volume of a unit hypercube with side length $s = 1$

Dimensions (d)	Volume (1^d)
2	1
3	1
10	1
50	1
100	1
500	1

Now consider a hypersphere (radius r)

$$V_d(r) = \frac{\pi^{d/2}}{\Gamma(\frac{d}{2} + 1)} r^d$$

$\propto r^d$

As dimensions increase, almost all the volume lies **near the edges** (far from the center).

DISTANCES CONVERGE IN HIGH DIMENSIONS

Dimensions (d)	Nearest Neighbor Distance (avg)	Farthest Neighbor Distance (avg)	Ratio (far/near)
2	0.21	1.41	6.7x
10	1.18	1.94	1.6x
50	6.10	6.95	1.1x
100	9.53	10.29	1.1x
500	22.36	22.87	1.02x

In 500D, the nearest and farthest points are almost **the same distance!**



WHY KNN FAILS IN HIGH DIMENSIONS



The concept of "close" loses meaning.

All points look equally far.



Distance metrics become less discriminative.

Weak neighborhood structure.



KNN relies on local neighborhoods.

In high-D, neighborhoods don't exist.

Result: KNN becomes random guessing.

REAL-WORLD IMPACT



Text Data (NLP)

→ Word embeddings often have 100s of dimensions (e.g., 300, 768).



Images (Vision)

→ Flattened pixels = tens of thousands of dimensions.



Result

→ KNN becomes slow, memory-heavy, and highly inaccurate.



Key Takeaway: As dimensions increase, distance loses meaning. In 500D space, all points are equally far apart, so KNN cannot find meaningful neighbors. That's why **KNN fails on text and vision data**.



Evaluating Classifiers

The Confusion Matrix



You cannot rely purely on “Accuracy” if the dataset is imbalanced.



True Positives (TP) / True Negatives (TN): Correct predictions.



False Positives (FP): Type I Error (False Alarm).



False Negatives (FN): Type II Error (Missed Detection).

CONFUSION MATRIX (BINARY CLASSIFICATION)

		Actual (Ground Truth)	
		Positive (1)	Negative (0)
Predicted (by Model)	Positive (1)	True Positive (TP) Correctly predicted as Positive	False Positive (FP) Incorrectly predicted as Positive (False Alarm)
	Negative (0)	False Negative (FN) Incorrectly predicted as Negative (Missed)	True Negative (TN) Correctly predicted as Negative

- **TP:** We predicted YES, and it was YES.
- **TN:** We predicted NO, and it was NO.
- **FP:** We predicted YES, but it was NO.
- **FN:** We predicted NO, but it was YES.

Rows = Model's Predictions | Columns = Actual Truth

WHY ACCURACY CAN BE MISLEADING

Example: Disease Detection
Out of 10,000 patients, only 100 have the disease (1%).

DUMB MODEL (Always predicts “No Disease”)



👤 = Has Disease (100) 👤 = No Disease (9,900)

Confusion Matrix:

TP = 0	FP = 0
FN = 100	TN = 9,900

$$\text{Accuracy} = \frac{(0 + 9,900)}{10,000} = 99.00\%$$

BETTER MODEL (Actually learns!)



👤 = Has Disease (100) 👤 = No Disease (9,900)

Confusion Matrix (Example):

TP = 70	FP = 90
FN = 30	TN = 9,810

$$\text{Accuracy} = \frac{(70 + 9,810)}{10,000} = 98.80\%$$



Accuracy is **higher** for the dumb model, but it is clinically **USELESS**. Always look at **TP**, **FP**, **FN**, and **TN**!

KEY METRICS DERIVED FROM THE CONFUSION MATRIX

Accuracy

Overall correctness.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Good only if classes are balanced.

Precision (PPV)

Of all positive predictions, how many were correct?

$$\text{Precision} = \frac{TP}{TP + FP}$$

High precision = Low false alarms.

Recall (TPR / Sensitivity)

Of all actual positives, how many did we catch?

$$\text{Recall} = \frac{TP}{TP + FN}$$

High recall = Low missed detections.

Specificity (TNR)

Of all actual negatives, how many did we catch?

$$\text{Specificity} = \frac{TN}{TN + FP}$$

High specificity = Low false alarms.

F1 Score

Harmonic mean of Precision and Recall.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Best single metric for imbalanced data.

EXAMPLE CONFUSION MATRIX AND METRICS

		Actual	
		Positive (1)	Negative (0)
Predicted	Positive (1)	75	25
	Negative (0)	15	885

- **Accuracy** = $(75 + 885) / 1,000 = 96.00\%$
- **Precision** = $75 / (75 + 25) = 75.00\%$
- **Recall** = $75 / (75 + 15) = 83.33\%$
- **Specificity** = $885 / (885 + 25) = 97.25\%$
- **F1 Score** = $2 \times (0.75 \times 0.8333) / (0.75 + 0.8333) = 0.789$

High accuracy, but recall < 100% → we are missing 15 positives!



Key Takeaway: The Confusion Matrix tells the story. Use it to understand the **types of errors** your model is making, not just whether it is “right” most of the time.







Precision, Recall, and F1-Score

The True Metrics of Success



• Precision:

Out of all predicted Positives, how many were actually Positive?

$$\text{Precision} = \frac{TP}{TP + FP}$$



• Recall:

Out of all actual Positives, how many did we find?

$$\text{Recall} = \frac{TP}{TP + FN}$$



• F1-Score:

The harmonic mean of Precision and Recall. Use this when your dataset is **uneven**.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

CONFUSION MATRIX (BINARY CLASSIFICATION)

		Actual (Ground Truth)	
		Positive (1)	Negative (0)
Predicted (by Model)	Positive (1)	True Positive (TP) We predicted YES, and it was YES.	False Positive (FP) We predicted YES, but it was NO.
	Negative (0)	False Negative (FN) We predicted NO, but it was YES.	True Negative (TN) We predicted NO, and it was NO.

- **TP:** Correct Positive (Hit)
- **TN:** Correct Negative (Correct Rejection)
- **FP:** Type I Error (False Alarm)
- **FN:** Type II Error (Missed Detection)

TYPE I vs TYPE II ERRORS

Type I Error (False Positive)

You cried wolf when there was no wolf.

(False Alarm)



Type II Error (False Negative)

You missed the wolf that was actually there.

(Missed Detection)



INTUITIVE UNDERSTANDING

Precision: Of the predicted positives...



Focuses on the **QUALITY** of positive predictions.

High precision = Few false alarms

Recall: Of the actual positives...



Focuses on the **COVERAGE** of actual positives.

High recall = Few missed detections

EXAMPLE CALCULATION

Suppose: **TP = 70**, **FP = 30**, **FN = 20**, **TN = 880**



$$\text{Precision} = \frac{TP}{TP + FP} = \frac{70}{70 + 30} = \mathbf{0.70} \text{ (70\%)}$$



$$\text{Recall} = \frac{TP}{TP + FN} = \frac{70}{70 + 20} = \mathbf{0.78} \text{ (78\%)}$$



$$F1 = 2 \times \frac{P \times R}{P + R} = 2 \times \frac{0.70 \times 0.78}{0.70 + 0.78} = \mathbf{0.74} \text{ (74\%)}$$

F1 balances Precision and Recall. It is high only when both are high.

WHY ACCURACY CAN MISLEAD (IMBALANCED DATA)

Disease screening example (1% have the disease)

Out of 10,000 patients:

Always predict **NEGATIVE**



👤 = No Disease (9,900) 👤 = Disease (100)

$$\text{Accuracy} = \frac{9,900}{10,000} = 99.00\%$$

But Recall = 0% (we missed all 100 sick patients!)

Better model



👤 = Correct 👤 = Missed (FN) 👤 = Other

Accuracy might drop, but Recall improves!

Precision, Recall, and F1 tell the real story.



Key Takeaway:

Accuracy can be deceptive when classes are imbalanced. **Precision** penalizes false alarms. **Recall** penalizes missed detections. **F1-Score** balances both. Use F1-Score as your go-to metric when your dataset is **uneven**.

